**CURSO DE PÓS-GRADUAÇÃO EM CIÊNCIA DE DADOS E INTELIGÊNCIA ARTIFICIAL**

**GIOVANE LUNA MIRANDA**

# APLICAÇÃO DE ARQUITETURAS RETRIEVAL-AUGMENTED GENERATION (RAG) NA ENGENHARIA DE REQUISITOS: UMA PROPOSTA DE FRAMEWORK PARA APOIO ÀS ATIVIDADES DO PROCESSO DE REQUISITOS

*Proposta de Trabalho de Conclusão de Curso (TCC)*

**DOURADOS - MS**

**2026**

# 1. INTRODUÇÃO

Nas últimas décadas, a Engenharia de Software consolidou-se como campo essencial da Computação, impulsionada pela complexidade dos sistemas modernos e pela dependência das empresas em relação a soluções digitais confiáveis. Construir software hoje não é apenas programar; é uma atividade multidisciplinar que envolve metas de negócio, fatores humanos e escolhas estratégicas. Por isso, adotar metodologias bem definidas tornou-se vital para garantir que o produto final realmente atenda às necessidades de quem o utiliza.

A Engenharia de Requisitos é a disciplina responsável por fazer a ponte entre o problema do cliente e a solução técnica. Como bem apontam Sommerville (2019) e Pressman e Maxim (2020), erros ou omissões nessa fase inicial comprometem todo o projeto, provocando atrasos, estouros de orçamento e retrabalho em etapas posteriores — desde a definição da arquitetura até a manutenção.

Na prática, gerenciar esse ciclo de vida (que engloba elicitação, análise, especificação, validação, controle de mudanças e rastreabilidade) é difícil. Ruídos de comunicação com stakeholders, a dificuldade em registrar o conhecimento informal e a constante mudança de escopo tornam o fluxo complexo e vulnerável a erros.

Recentemente, a Inteligência Artificial (IA), especialmente os Modelos de Linguagem de Grande Porte (LLMs), trouxe novas possibilidades para apoiar o trabalho do analista. O uso prático de IA generativa já se mostra viável para resumir documentos, sugerir e validar requisitos, identificar inconsistências conceituais e escrever casos de uso.

Pesquisas recentes sugerem que os LLMs reduzem o tempo gasto em tarefas repetitivas e auxiliam na escrita de documentação técnica. Apesar disso, esses modelos trazem desafios estruturais importantes. Como se baseiam em dados históricos de treinamento, eles podem gerar informações falsas ou desatualizadas. Outro ponto crítico é a falta de acesso a documentos internos e sigilosos, o que limita seu uso em cenários corporativos regidos por regras muito específicas.

Para contornar essas barreiras, o paradigma de Geração Aumentada por Recuperação (RAG) integra modelos gerativos a sistemas de busca de dados externos. Em vez de depender apenas da memória interna obtida no treinamento, o modelo consulta uma base externa de documentos para extrair trechos relevantes, gerando respostas mais precisas e fáceis de auditar.

A aplicação do RAG vem crescendo em várias áreas, inclusive na Engenharia de Software. Variantes mais avançadas da arquitetura tradicional — como GraphRAG, Self-RAG, Corrective RAG (CRAG), Adaptive RAG, RAPTOR e Agentic RAG — utilizam estruturas sofisticadas de indexação e verificação que vão muito além da busca semântica básica.

No entanto, embora a pesquisa sobre IA e as variações de RAG avancem rápido, nota-se uma lacuna: falta investigar de forma comparativa como cada uma dessas arquiteturas se comporta nas diferentes tarefas da Engenharia de Requisitos. Faltam guias práticos que orientem profissionais e pesquisadores a escolher a melhor variante de RAG para necessidades específicas, seja para elicitar requisitos, validar regras de consistência ou mapear a rastreabilidade.

Diante desse cenário, esta pesquisa analisa a aplicação e o desempenho de diferentes arquiteturas de RAG nas atividades do processo de requisitos. O intuito é formular um framework de tomada de decisão que guie a escolha do modelo ideal para cada tarefa, promovendo uma integração metodológica mais segura no desenvolvimento de sistemas.

# 2. FUNDAMENTAÇÃO TEÓRICA

## 2.1 Engenharia de Software

A Engenharia de Software estuda e aplica práticas, métodos e princípios para construir sistemas de forma estruturada. A meta central é entregar sistemas de alta qualidade respeitando limites de custo, prazo e facilidade de manutenção futura.

Sommerville (2019) descreve a área como uma disciplina focada em processos que guiam a criação, a operação e a evolução de sistemas. Para o autor, o software precisa ser tratado com o rigor de uma engenharia tradicional, exigindo planejamento detalhado, documentação, testes contínuos e melhoria constante para garantir sua confiabilidade.

Na mesma linha, Pressman e Maxim (2020) enxergam a disciplina como um conjunto de boas práticas voltadas ao desenvolvimento organizado e previsível. Eles destacam que desenvolver software vai muito além de codificar, englobando também a gestão de projetos, a garantia de qualidade e a comunicação entre todas as pessoas envolvidas.

Sob essa ótica, programar torna-se apenas uma etapa em um fluxo de atividades interligadas. Por mais que os ciclos de vida variem — sejam modelos tradicionais, iterativos ou ágeis —, todos dependem da correta compreensão do problema de negócio e das expectativas do cliente.

É aqui que reside a importância da Engenharia de Requisitos. Entender e registrar o que o cliente de fato precisa determina o sucesso de qualquer projeto, não importa a metodologia utilizada. Quando ocorrem falhas ou contradições nessa fase, elas tendem a se espalhar pelas etapas seguintes, tornando correções futuras muito mais difíceis, demoradas e caras.

Estudos da literatura confirmam que boa parte dos projetos de software fracassa justamente devido a requisitos incompletos, ambíguos ou mal interpretados. Por isso, investir em técnicas consistentes de requisitos não traz apenas ganhos imediatos de qualidade, mas é crucial para mitigar riscos, evitar retrabalho e controlar o custo de evolução do sistema.

## 2.2 Engenharia de Requisitos

A Engenharia de Requisitos lida com métodos para identificar, analisar, documentar e acompanhar as características de um sistema ao longo do tempo. Segundo Wiegers e Beatty (2013), um requisito representa uma condição ou capacidade necessária para resolver um problema de negócio ou satisfazer o usuário.

Esse esforço vai muito além de preencher documentos. O objetivo principal é construir um entendimento comum entre clientes, usuários, analistas e a equipe de desenvolvimento, reduzindo dúvidas na interpretação das regras e otimizando a comunicação do projeto.

A literatura divide o processo em seis etapas básicas:

1. **Elicitação**: Consiste em descobrir o que o negócio precisa e o que os usuários esperam. O analista mapeia objetivos e fluxos usando entrevistas, workshops, observações e análise de documentos existentes. Como as informações costumam estar dispersas entre diferentes setores da empresa, a etapa exige bastante habilidade de comunicação para mediar conflitos de interesse.
2. **Análise**: Com os dados em mãos, os requisitos são organizados, detalhados e priorizados. Esse filtro ajuda a detectar contradições, dados repetidos ou inviabilidades técnicas, além de guiar acordos com os stakeholders.

3. **Especificação**: É a documentação formal e clara do comportamento esperado do sistema. Dependendo do modelo de desenvolvimento, pode ser feita por meio de documentos formais de requisitos (SRS), diagramas UML ou, em ambientes ágeis, histórias de usuário e critérios de aceitação.

4. **Validação**: Serve para confirmar se as especificações atendem de fato ao que o cliente precisa. Revisões, protótipos rápidos e reuniões de alinhamento são comuns aqui para evitar que códigos comecem a ser escritos sobre premissas incorretas.

5. **Gerenciamento de Mudanças**: Os requisitos mudam à medida que o negócio evolui, novas normas surgem ou a visão do próprio cliente amadurece. Essa atividade cuida de avaliar o impacto de alterações no projeto, garantindo que o cronograma e os artefatos técnicos continuem alinhados.

6. **Rastreabilidade**: É a capacidade de ligar os requisitos às regras de negócio, códigos e casos de teste. Esse mapeamento simplifica o diagnóstico de impacto quando algo muda e apoia auditorias, sendo vital para sistemas complexos ou regulados.

Embora essas práticas estejam bem consolidadas, executá-las depende muito da experiência do analista e do acesso a dados organizacionais. Por essa razão, pesquisadores e profissionais buscam na Inteligência Artificial uma forma de apoiar esse processo e otimizar a tomada de decisões, como veremos adiante.

## 2.3 Inteligência Artificial Aplicada à Engenharia de Requisitos

Os avanços recentes na Inteligência Artificial causaram um impacto direto no desenvolvimento de software. A popularização dos Modelos de Linguagem de Grande Porte (LLMs) expandiu as possibilidades de automação de tarefas baseadas em texto, abrindo caminho para a aplicação dessas tecnologias no ciclo de requisitos.

Historicamente, ler grandes volumes de documentação, redigir especificações e validar a consistência das regras exigem um esforço intelectual intenso, muito atrelado à experiência do profissional. A busca por ferramentas inteligentes visa reduzir essa carga operacional sem comprometer a qualidade dos artefatos produtos.

A literatura aponta usos práticos da IA em várias frentes: classificar requisitos automaticamente (funcionais e não funcionais), extrair necessidades de textos brutos, gerar histórias de usuário e detectar ambiguidades. Modelos generativos também atuam como assistentes na elicitação, ajudando o analista a formular perguntas complementares e a identificar brechas de informação ao conversar com clientes.

Dalpiaz, Franch e Horkoff (2018) destacam que a Engenharia de Requisitos se apoia na gestão de conhecimento e na comunicação — duas áreas em que os modelos de linguagem, treinados em síntese e compreensão textual, se destacam. Revisões sistemáticas da literatura mostram que o interesse acadêmico sobre o tema disparou com o surgimento dos modelos fundacionais e generativos.

Para empresas com grandes volumes de dados legados, essas ferramentas prometem mais agilidade na leitura e análise de relatórios, detecção precoce de erros e apoio à tomada de decisão. Contudo, a literatura aponta limitações críticas. A primeira delas é que o conhecimento do modelo é estático (limitado à data em que foi treinado), o que causa obsolescência rápida. Além disso, há o risco de alucinação, quando a IA gera respostas gramaticalmente corretas, mas falsas ou incorretas do ponto de vista prático.

Outra barreira dos LLMs é lidar com documentos extensos e variados, como contratos, regulamentos e especificações antigas. Mesmo quando esses arquivos são enviados diretamente ao modelo, os limites físicos de sua janela de contexto (*context window*) reduzem a quantidade de dados processada de uma única vez, prejudicando a profundidade da análise.

Fica claro que usar LLMs puros e de forma isolada não basta em cenários que exigem alta rastreabilidade e fidelidade estrita às regras organizacionais. É para suprir essa brecha que surgem abordagens focadas em recuperação dinâmica de informações, com destaque para a Geração Aumentada por Recuperação (RAG).

### 2.4 Retrieval-Augmented Generation (RAG)

Proposto inicialmente por Lewis et al. (2020), o RAG acopla modelos gerativos a bases de informação externas. Em vez de formular respostas apoiando-se apenas nas informações absorvidas durante seu treinamento, o modelo pesquisa em repositórios de documentos e usa as partes recuperadas como contexto, gerando respostas mais precisas e fáceis de checar.

No RAG tradicional, a pergunta do usuário é convertida em um vetor matemático de alta dimensão (*embedding*), que carrega as propriedades semânticas do texto. Esse vetor consulta um banco de dados vetorial (*vector database*), onde estão armazenados os trechos segmentados dos documentos organizacionais.

Os fragmentos mais relevantes são anexados à mensagem original enviada ao LLM. O modelo então elabora a resposta combinando as informações da base externa com a sua capacidade linguística. Esse processo reduz as alucinações e mantém as respostas alinhadas aos documentos oficiais da empresa.

O sucesso desse fluxo depende de vários componentes integrados:

- **Segmentação (*chunking*)**: Divide os documentos originais em blocos menores. O tamanho desses blocos e a sua sobreposição (*overlap*) devem ser balanceados; partes muito pequenas fragmentam o sentido do texto, enquanto partes muito grandes trazem informações desnecessárias.
- **Representação Semântica (*embeddings*)**: Converte palavras e parágrafos em vetores numéricos. Modelos como Sentence Transformers ou APIs comerciais (como OpenAI) realizam essa conversão, permitindo buscas conceituais que vão além de palavras-chave simples.

- **Armazenamento e Busca**: Ferramentas especializadas como FAISS, ChromaDB, Milvus, Weaviate e Pinecone guardam esses vetores e localizam rapidamente os blocos mais próximos da consulta usando algoritmos de busca de vizinhos mais próximos (ANN).

Para refinar as respostas, arquiteturas avançadas incluem processos de reordenação (*re-ranking*). Modelos secundários (como *cross-encoders*) reavaliam a proximidade dos blocos com a pergunta do usuário, reorganizando a lista para priorizar trechos mais ricos antes de enviá-los ao gerador final.

Além do modelo clássico, a literatura recente propõe variantes projetadas para tarefas complexas. O GraphRAG (EDGE et al., 2024), desenvolvido pela Microsoft, utiliza grafos de conhecimento para conectar conceitos e entidades, facilitando análises holísticas e que exigem rastreabilidade de dados. O Self-RAG (ASAI et al., 2024) adota regras de autoavaliação, permitindo que a própria IA decida se precisa buscar novos blocos ou se o texto gerado é satisfatório. Já o Corrective RAG (CRAG) (YAN et al., 2024) avalia os trechos trazidos da base e, se forem insuficientes, faz consultas externas na internet para complementar as informações.

Outras variantes buscam otimizar custos e estrutura de dados. O Adaptive RAG (SARAVANAN et al., 2024) categoriza a dificuldade da pergunta para escolher a melhor estratégia de busca, economizando recursos computacionais. O Agentic RAG divide requisições complexas em etapas executadas por agentes autônomos. E o RAPTOR (SAHA et al., 2024) cria resumos hierárquicos em árvore, agilizando buscas que precisam de uma visão geral sobre múltiplos arquivos longos.

Embora cada variante de RAG tenha pontos fortes claros — como grafos para rastrear dados ou agentes para decisões iterativas —, a literatura atual ainda não investigou como essas diferentes abordagens podem ser alinhadas sistematicamente às tarefas específicas da Engenharia de Requisitos.

## 3. IDENTIFICAÇÃO DA LACUNA CIENTÍFICA

A pesquisa acadêmica mostra um volume crescente de estudos sobre o uso de IA no processo de requisitos. Trabalhos recentes analisam o emprego de aprendizado de máquina e LLMs para classificar dados, extrair conceitos, sugerir histórias de usuário, identificar ambiguidades e apoiar escolhas técnicas.

Ao mesmo tempo, o ecossistema RAG evoluiu de forma expressiva para resolver fraquezas inerentes aos modelos puros. Surgiram recursos como o mapeamento em grafos do GraphRAG, os ciclos de autoavaliação do Self-RAG e a recuperação baseada em agentes autônomos.

Embora ambas as frentes tragam avanços significativos, elas parecem caminhar de forma isolada. Os estudos em requisitos focam em usar a IA pontualmente em tarefas manuais clássicas. Já as pesquisas em RAG priorizam melhorar a engenharia de prompts e os fluxos de indexação. Com essa falta de integração, quase não há estudos que tracem um paralelo estruturado entre os recursos técnicos de cada variante de RAG e as necessidades específicas do analista de requisitos.

Essa separação ignora o fato de que cada etapa de requisitos tem características únicas. A elicitação é interativa por natureza e depende de diálogos constantes com o cliente. Por outro lado, a rastreabilidade exige conexões formais entre requisitos, códigos e testes. Já o gerenciamento de mudanças demanda atualizações dinâmicas na documentação e análise de impacto. Com necessidades tão distintas, uma arquitetura RAG única e genérica dificilmente dará conta de todas as fases do processo de requisitos.

Outra limitação é que os testes de arquiteturas RAG descritos na literatura costumam focar em buscas simples de perguntas e respostas ou assistência básica de código. Eles não levam em conta os fluxos de trabalho e o rigor técnico que a

engenharia de software exige. Pesquisadores e desenvolvedores não têm parâmetros sólidos para definir qual arquitetura escolher ao planejar a integração de RAG no ciclo de vida de desenvolvimento.

Falta, portanto, um estudo sistemático que cruze as variações de RAG com o ciclo de requisitos. Criar um framework com esse foco preenche esse espaço vazio, gerando contribuições teóricas e práticas tanto para a academia quanto para o mercado.

## 4. PROPOSTA

Com base nesse cenário, este trabalho avalia como as diferentes variantes de RAG podem apoiar a Engenharia de Requisitos, propondo um framework metodológico para guiar a escolha da melhor arquitetura para cada atividade.

O estudo tem caráter analítico e busca categorizar as arquiteturas RAG descritas na literatura acadêmica, em vez de criar um algoritmo inédito. O objetivo principal é cruzar as propriedades técnicas de cada variação de RAG com os desafios operacionais do analista de requisitos.

Para alcançar isso, a pesquisa será realizada nas seguintes etapas:

1. **Revisão da literatura**: Mapeamento do estado da arte sobre Engenharia de Requisitos, IA aplicada e arquiteturas RAG, buscando consolidar o conhecimento científico já validado.
2. **Mapeamento de características**: Análise das propriedades técnicas das variantes RAG (como o uso de grafos, autoavaliação, agentes e roteamento dinâmico).
3. **Pareamento das atividades**: Cruzamento desses perfis técnicos com as restrições das seis macroatividades de requisitos.

A síntese desses passos dará origem ao framework de suporte à decisão. Longe de propor uma receita única, a ideia é fornecer critérios claros para ajudar na seleção tecnológica, considerando a natureza das bases de dados, a complexidade da tarefa e os limites de cada etapa de engenharia.

A Tabela 1 traz um mapeamento preliminar dessas relações, que será validado no decorrer da pesquisa.

**Tabela 1. Associação preliminar entre etapas de requisitos e variantes de RAG**

| ETAPA DA ENGENHARIA DE REQUISITOS | ARQUITETURA POTENCIALMENTE ADEQUADA | JUSTIFICATIVA |
|---|---|---|
| **Elicitação** | Agentic RAG | Permite interação iterativa, divisão de problemas em etapas menores e suporte ativo na descoberta de novos requisitos em entrevistas. |
| **Análise** | Self-RAG / Corrective RAG | Favorece a revisão crítica, ajudando a identificar inconsistências conceituais e a validar as informações extraídas. |
| **Especificação** | Hierarchical RAG (RAPTOR) | Facilita a leitura e navegação em bases documentais longas, auxiliando a resumir dados dispersos. |
| **Validação** | Corrective RAG | Auxilia na verificação de conformidade ao cruzar os requisitos gerados com documentos de referência e regulamentos. |
| **Gerenciamento de Mudanças** | Adaptive RAG | Ajusta dinamicamente a profundidade da busca de acordo com o contexto e a complexidade das alterações pedidas. |
| **Rastreabilidade** | GraphRAG | Representa as conexões e interdependências entre requisitos, regras de |

| ETAPA DA ENGENHARIA DE REQUISITOS | ARQUITETURA POTENCIALMENTE ADEQUADA | JUSTIFICATIVA |
| --- | --- | --- |
| | | negócio, testes e blocos de código usando grafos. |

Essa associação serve como hipótese principal do estudo. A validação dessa proposta guiará o desenvolvimento do trabalho, sendo sustentada por fundamentação teórica sólida e, se necessário, testes exploratórios com estudos de caso reais.

Por fim, vale ressaltar que o framework não substitui o papel do analista de requisitos. A ferramenta servirá como um apoio metodológico para dar mais transparência e segurança à adoção de IA generativa em projetos corporativos.

## 5. POSSÍVEIS CONTRIBUIÇÕES CIENTÍFICAS

A IA generativa veio para ficar no desenvolvimento de software. Porém, as empresas ainda adotam essa tecnologia de forma experimental (*ad-hoc*), sem muita estrutura. Isso se acentua na Engenharia de Requisitos, que exige muita troca de informação, interpretação de regras tácitas e leitura de bases de dados variadas.

Diante disso, a principal contribuição deste trabalho é estruturar um modelo que cruze as variações de RAG com as etapas de requisitos. Ao unificar conceitos hoje dispersos em artigos acadêmicos, a pesquisa oferece uma base fundamentada para guiar a adoção dessas ferramentas no mercado.

Do lado acadêmico, o estudo aproxima o processamento de linguagem natural da engenharia de requisitos. O framework poderá servir de ponto de partida para estudos empíricos e experimentos sobre o uso prático de assistentes inteligentes em ambientes empresariais.

Na prática, o trabalho ajuda analistas, arquitetos de software e gestores no planejamento dessas tecnologias. Em vez de implementar o RAG clássico para qualquer problema, os times de engenharia poderão avaliar o perfil da tarefa, o volume de documentos e a necessidade de rastreabilidade para escolher a arquitetura ideal.

Com isso, as empresas reduzem o risco de projetos de IA malsucedidos. O uso de critérios claros ajuda a manter a consistência dos requisitos e a aproveitar melhor o conhecimento acumulado em documentações antigas.

Como resultado, a escolha correta do RAG pode mitigar conflitos em requisitos, agilizar pesquisas conceituais complexas e dar mais confiabilidade à validação e garantia de qualidade do software final. Espera-se também que este estudo sirva como pontapé inicial para criar diretrizes de governança e segurança na automação de processos de engenharia.

## 6. CONSIDERAÇÕES FINAIS

A Engenharia de Requisitos continua sendo um pilar indispensável do desenvolvimento de software. Mesmo com metodologias de desenvolvimento consolidadas, problemas crônicos como falhas na elicitação, desvios de escopo e dificuldades de rastreabilidade ainda ocorrem em projetos de todos os portes.

Os LLMs trouxeram novas formas de atenuar esses gargalos. A capacidade de entender e produzir textos em linguagem natural viabiliza ferramentas que poupam trabalho braçal, elevam a produtividade e apoiam as decisões sobre a especificação do software.

Mas a literatura deixa claro que usar esses modelos isoladamente não atende aos requisitos de rigor e segurança das empresas. A IA pura esbarra na ausência de contexto de negócio, no risco de gerar dados falsos (alucinações) e na obsolescência das respostas. O RAG ajuda a resolver isso ao conectar a geração de texto à busca dinâmica de dados corporativos atualizados.

Embora a computação venha criando várias derivações de RAG, há poucos estudos avaliando como usá-las estrategicamente para resolver os problemas típicos das atividades de requisitos.

Esta proposta de TCC visa preencher essa brecha. Ao mapear o comportamento das variantes de RAG diante das demandas clássicas de requisitos, formularemos um framework de apoio à escolha arquitetural. O estudo busca encurtar a distância entre a pesquisa em IA e a Engenharia de Software, servindo como base sólida para o desenvolvimento deste projeto e de pesquisas futuras na área.

## REFERÊNCIAS


ASAI, Akari et al. Self-RAG: Learning to Retrieve, Generate and Critique through Self-Reflection. In: **International Conference on Learning Representations (ICLR)**, 2024.

DALPIAZ, Fabiano; FRANCH, Xavier; HORKOFF, Jennifer. Goal-Oriented Requirements Engineering: A Guided Tour. In: **IEEE International Requirements Engineering Conference Workshops**, 2018.

EDGE, Darren et al. From Local to Global: A GraphRAG Approach to Query-Focused Summarization. **Microsoft Research**, 2024.

GAO, Yunfan et al. Retrieval-Augmented Generation for Large Language Models: A Survey. **arXiv preprint arXiv:2312.10997**, 2023.

IEEE COMPUTER SOCIETY. **Guide to the Software Engineering Body of Knowledge (SWEBOK Guide V4.0)**. IEEE, 2024.

KOJIMA, Takeshi et al. Large Language Models are Zero-Shot Reasoners. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 35, 2022.

LEWIS, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: **Advances in Neural Information Processing Systems (NeurIPS)**, v. 33, p. 9459–9474, 2020.

NAIDU, Rahul et al. Retrieval-Augmented Generation for Large Language Models: A Survey. **arXiv preprint arXiv:2404.14819**, 2024.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Software Engineering: A Practitioner's Approach**. 9. ed. New York: McGraw-Hill, 2020.

SAHA et al. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In: **International Conference on Learning Representations (ICLR)**, 2024.

SARAVANAN et al. Adaptive Retrieval-Augmented Generation: Foundation Models Meet Adaptive Retrieval. **arXiv preprint arXiv:2403.14481**, 2024.

SOMMERVILLE, Ian. **Software Engineering**. 10. ed. Boston: Pearson, 2019.

WIEGERS, Karl; BEATTY, Joy. **Software Requirements**. 3. ed. Redmond: Microsoft Press, 2013.

YAN, Sheng et al. Corrective Retrieval-Augmented Generation (CRAG). **arXiv preprint arXiv:2401.15884**, 2024.

YAO, Shunyu et al. ReAct: Synergizing Reasoning and Acting in Language Models. In: **International Conference on Learning Representations (ICLR)**, 2023.